KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Digital Note Organizer with File Storage

Submitted by

RAMALIDURGA M

25BCS045

Under the Guidance of

Dr. BOOPALAN SUNDARAMOORTHY

Technical Head

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Digital Note Organizer with File Storage"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING LANGUAGE**, is a **record of my original work** carried out under the guidance of , **Dr.S.BOOPALAN SUNDARAMOORTHY** , Technical Head , Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

RAMALIDURGA M

CERTIFICATE

This is to certify that the project entitled **"Digital Note Organizer with File Storage"** is a **bona fide work** carried out by **RAMALIDURGA M, I-'A'(2025-2026)**, **Department of Computer Science**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING LANGUAGE**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date: 10/04/2026

Submitted for the Viva-Voce Examination held on 10/04/2026

Guide:
(Technical Head, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Digital Note Organizer with File Storage** is a Java-based application designed to help users create, manage, and store digital notes efficiently. In many situations, people store important information in notebooks or unorganized digital files, which makes it difficult to manage and retrieve the information when needed. This project provides a simple solution by offering a digital platform to organize notes in a structured and accessible way.

The system allows users to create new notes, view saved notes, edit existing notes, and delete notes that are no longer required. All notes are stored using file storage so that the data remains available even after the program is closed. The application ensures that users can easily manage their notes without losing important information.

The project is developed using **Core Java programming concepts** such as classes, objects, file handling, loops, and conditional statements. The application can be implemented as a console-based system or extended with a graphical user interface using Java Swing.

The main objective of this system is to provide a simple and efficient digital tool for organizing notes and storing files. This project demonstrates the practical implementation of Java programming in developing useful real-world applications for personal and academic use.

1. INTRODUCTION

In today's digital era, people frequently need to store important information such as ideas, reminders, study materials, meeting notes, and personal records. Traditionally, many people use paper notebooks or scattered digital documents to keep such information. However, these methods can make it difficult to organize, manage, and retrieve information when needed. Paper notes can be lost or damaged, and unorganized digital files can make searching for specific information time-consuming.

A **Digital Note Organizer** provides a convenient solution for managing notes in an organized and efficient manner. It allows users to create, store, view, edit, and delete notes digitally. By storing notes in a structured format, users can easily access their information whenever required.

The **Digital Note Organizer with File Storage** is a Java-based application designed to help users manage their notes effectively. The system allows users to create new notes and save them using file storage so that the data remains stored even after the program is closed. Users can also view previously saved notes, update the content of existing notes, and remove notes that are no longer needed.

This project is developed using **Core Java programming concepts** such as classes, objects, methods, loops, conditional statements, and file handling. The application can be implemented as a simple console-based program and can also be extended in the future by adding graphical user interfaces or database storage.

The main goal of this project is to provide a simple digital tool for organizing notes and storing information efficiently. It also demonstrates how Java programming can be used to develop practical applications that solve real-world problems.

1.1 Objective

The main objective of the **Digital Note Organizer with File Storage** is to develop a simple and efficient application using Java that helps users create, store, and manage digital notes in an organized manner.

The specific objectives of this project are:

- To design and develop a **Java-based digital note organizer**.
- To allow users to **create and save notes digitally**.
- To provide an option to **view previously saved notes** easily.
- To allow users to **edit or update existing notes** when required.
- To provide the ability to **delete notes that are no longer needed**.
- To store notes using **file storage so that the data remains saved permanently**.
- To demonstrate the use of **Core Java programming concepts** such as classes, objects, loops, conditional statements, and file handling.

1.2 Overview

The **Digital Note Organizer with File Storage** is a menu-driven application developed using the Java programming language. The system provides a simple digital platform where users can create, manage, and organize their notes efficiently.

The application allows users to create new notes and store them as files so that the information is saved permanently. Users can also view previously saved notes, edit the content of existing notes, and delete notes that are no longer required. This helps users keep their information organized and easily accessible.

The system uses **Java file handling** to store notes in files, ensuring that the data remains available even after the application is closed. The menu-driven interface makes it easy for users to navigate through different options such as creating, viewing, editing, and deleting notes.

This project demonstrates the practical use of **Core Java programming concepts**, including classes, objects, methods, loops, conditional statements, and file handling. The application can be implemented as a console-based program and can be further enhanced by adding graphical user interfaces, database connectivity, and advanced search features in the future.

1.3 Features

The **Digital Note Organizer with File Storage** includes the following main features:

• **Create Notes** – Allows users to create and save new notes digitally.

• **View Notes** – Enables users to view previously saved notes whenever required.

• **Edit Notes** – Allows users to modify or update the content of existing notes.

• **Delete Notes** – Provides the option to remove notes that are no longer needed.

• **File Storage** – Stores notes using file handling so that the data remains saved even after the program is closed.

• **User-Friendly Interface** – Provides a simple menu-driven interface that is easy for users to operate.

• **Efficient Note Management** – Helps users organize and manage their notes in a structured and convenient way.

2. PROGRAM ANALYSIS

Program analysis is an important stage in software development where the problem is studied carefully and the requirements of the system are identified. This phase helps in understanding how the system should work and what features are needed to solve the problem effectively.

In the case of the **Digital Note Organizer with File Storage**, the analysis focuses on understanding the need for a digital system that allows users to manage their notes efficiently. Many users face difficulties in organizing and retrieving information when notes are stored in notebooks or scattered digital files.

During this stage, the problems of the existing system are identified and a suitable computerized solution is proposed. The proposed system allows users to create, store, view, edit, and delete notes easily using a simple Java-based application.

The program analysis also helps in identifying the required functionalities such as note creation, file storage, note retrieval, and note management. By analyzing these requirements, the system can be designed in a way that ensures efficiency, simplicity, and ease of use for the users.

2.1 Problem Definition

In many situations, people store important information such as reminders, ideas, study materials, and personal notes in paper notebooks or unorganized digital files. This traditional method creates several challenges when trying to manage and retrieve information quickly.

Some common problems include:

- Difficulty in organizing large numbers of notes
- Time-consuming process to find specific information
- Risk of losing important notes when stored on paper
- Difficulty in editing or updating existing notes
- Lack of a structured system for storing and managing notes

When notes are stored in different places, it becomes difficult for users to maintain proper records and access the information when needed.

Therefore, there is a need for a **Digital Note Organizer** that allows users to store, manage, and retrieve their notes easily. The **Digital Note Organizer with File Storage** is designed to solve these problems by providing a simple Java-based application where users can create, view, edit, and delete notes while storing them securely using file storage.

2.2 Project Overview

The **Digital Note Organizer with File Storage** is a Java-based application developed to help users organize and manage their notes efficiently. The system provides a simple and user-friendly platform where users can create, store, view, edit, and delete notes according to their needs.

The application uses a menu-driven interface that allows users to choose different options such as creating a new note, viewing saved notes, editing existing notes, and deleting unwanted notes. All notes are stored using Java file handling, which ensures that the data is saved permanently and can be accessed later whenever required.

The system helps users keep their information organized and easily accessible. By storing notes digitally, the application reduces the risk of losing important information and makes it easier to manage large numbers of notes.

This project demonstrates the practical implementation of **Core Java programming concepts**, including classes, objects, loops, conditional statements, and file handling. The application can be further improved by adding features such as graphical user interfaces, search options, and database integration.

2.3 Scopes and Limitations

Scope

The Digital Note Organizer with File Storage provides a simple and efficient system for managing digital notes. The scope of this project includes the following:

• Creating and saving digital notes.
• Viewing previously stored notes.
• Editing and updating existing notes.
• Deleting notes that are no longer required.
• Storing notes using file storage for future access.
• Demonstrating the practical use of Java programming

concepts. This system can be used by:

• Students for managing study notes
• Teachers for storing important information
• Office employees for recording tasks or reminders
• Personal users for organizing daily notes

Limitations

Although the system provides basic note management features, it has some limitations:

- **The system uses local file storage only and does not use a database.**
- **It is designed for single-user access.**
- **The system does not include advanced search or filtering features.**
- **It does not provide cloud storage or online synchronization.**

These limitations can be improved in future versions by adding database connectivity, cloud storage, and advanced search features.

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation of the program. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Digital Note Organizer with File Storage** is designed as a menu-driven application where users can create, view, edit, and delete notes easily. The system stores notes using file storage so that the data remains saved even after the application is closed.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the application, such as creating notes, viewing stored notes, editing notes, deleting notes, and storing files.

This modular design helps in making the program more organized, easier to understand, and easier to maintain. By separating the system into different modules, each part of the program can perform its task efficiently and improve the overall functionality of the system.

3.1 Modules

The **Digital Note Organizer with File Storage** is divided into several modules to perform different functions of the system. Each module handles a specific task to ensure the smooth operation of the application.

1. Create Note Module

This module allows the user to create a new note. The user can enter the title and content of the note, which will then be saved as a file in the system.

Functions of this module include:

• Accepting note title and content from the user
• Saving the note using file storage
• Storing the note for future access

---

2. View Notes Module

This module allows users to view previously saved notes. The system reads the stored files and displays the content of the selected note.

Functions of this module include:

• Displaying the list of saved notes
• Allowing the user to select a note
• Displaying the content of the selected note

---

3. Edit Note Module

This module allows users to modify or update existing notes. The user can open a saved note, make changes to the content, and save the updated version.

Functions include:

• Selecting the note to edit
• Updating the content of the note
• Saving the updated note

---

4. Delete Note Module

This module allows users to delete notes that are no longer needed.

Functions include:

• Selecting the note to delete
• Removing the note file from storage

---

5. File Storage Module

This module manages the storage of notes using Java file handling. It ensures that all notes are saved properly and can be accessed later.

Functions include:

• Saving notes to files
• Reading notes from files
• Managing stored note data

---

6. Exit Module

This module allows the user to exit the application safely. When the user selects the exit option, the program terminates.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Digital Note Organizer with File Storage**, a structured development approach was followed to ensure that the system meets the user requirements and functions effectively.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed carefully. The need for a system that allows users to store and manage notes digitally is identified.

The main problems found in traditional note management systems include:

- Difficulty in organizing notes
- Risk of losing important information
- Time-consuming process to search for notes
- Difficulty in editing or updating notes

Based on these problems, the requirements of the Digital Note Organizer system were identified.

4.2. System Design

In this stage, the structure of the system is planned before coding begins. The system is divided into modules such as:

• Create Note Module
• View Notes Module
• Edit Note Module
• Delete Note Module
• File Storage Module
• Exit Module

This design helps in organizing the application and making it easier to implement.

4.3. Program Development

In this phase, the system is implemented using the **Java programming language**. The program is developed using a menu-driven approach that allows users to select different options for managing notes.

Core Java concepts used include:

• Classes and objects
• Methods
• Loops
• Conditional statements
• File handling

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases are used to verify the system functions, such as:

Test Case	Expected Result
Create note	Note saved successfully
View note	Stored note displayed
Edit note	Note updated correctly
Delete note	Note removed successfully

Testing helps ensure that the application works without errors.

4.5. Result and Documentation

After testing, the results of the system are documented. The application successfully performs note creation, storage, editing, and deletion.

The results are demonstrated through program execution and screenshots showing the input and output of the system.

Proper documentation is prepared to explain the design, implementation, and working of the system.

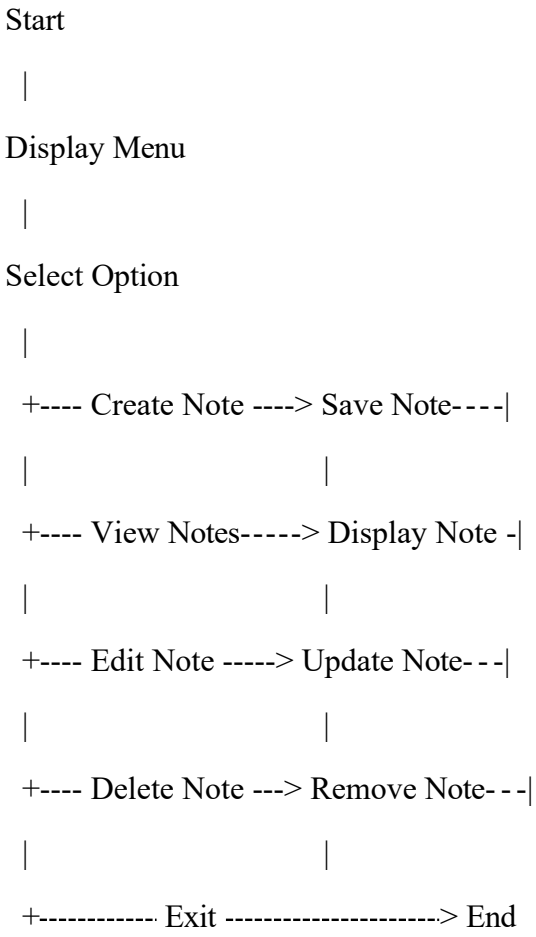
4.6. Algorithm

Algorithm for the **Digital Note Organizer with File Storage**:

1. Start the program.
2. Display the main menu with available options.
3. Ask the user to select an option from the menu.

4. If the user selects **Create Note**, enter the note title and content, then save the note to a file.
5. If the user selects **View Notes**, display the list of saved notes and show the selected note content.
6. If the user selects **Edit Note**, choose the note to edit and update its content, then save the changes.
7. If the user selects **Delete Note**, select the note and remove it from storage.
8. If the user selects **Exit**, terminate the program.
9. Repeat the menu until the user chooses the exit option.
10. End the program.

4.7. Flowchart



5. SOURCE CODE

```
import java.io.*;
import java.util.*;

class Note
   { int id;
   String title;
   String content;

   Note(int id, String title, String content) {
      this.id = id;
      this.title = title;
      this.content = content;
   }

   public String toString() {
      return id + "|" + title + "|" + content;
   }
}

public class DigitalNoteOrganizer {

   static final String FILE_NAME = "notes.txt";

   // Add Note
   public static void addNote(Scanner sc) throws IOException {
```

```
    FileWriter fw = new FileWriter(FILE_NAME, true);

    System.out.print("Enter Note ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Enter Title:
    "); String title = sc.nextLine();

    System.out.print("Enter Content: ");
    String content = sc.nextLine();

    Note note = new Note(id, title, content);
    fw.write(note.toString() + "\n");
    fw.close();

    System.out.println("Note added successfully!");
}

// View Notes
public static void viewNotes() throws IOException
    { File file = new File(FILE_NAME);

    if (!file.exists()) {
        System.out.println("No notes
        found."); return;
    }

    BufferedReader br = new BufferedReader(new FileReader(file));
```

```
    String line;

    System.out.println("\nAll Notes:");
    while ((line = br.readLine()) != null) {
        String[] parts = line.split("\\|");
        System.out.println("ID: " + parts[0]);
        System.out.println("Title: " + parts[1]);
        System.out.println("Content: " + parts[2]);
        System.out.println("----------------------");
    }
    br.close();
}

// Search Notes
public static void searchNotes(Scanner sc) throws IOException
    { File file = new File(FILE_NAME);

    if (!file.exists()) {
        System.out.println("No notes
        found."); return;
    }

    System.out.print("Enter keyword to search: ");
    String keyword = sc.nextLine();

    BufferedReader br = new BufferedReader(new FileReader(file));
    String line;
    boolean found = false;
```

```
    while ((line = br.readLine()) != null) {
        if (line.toLowerCase().contains(keyword.toLowerCase()))
            { String[] parts = line.split("\\|");
            System.out.println("ID: " + parts[0]);
            System.out.println("Title: " + parts[1]);
            System.out.println("Content: " +
            parts[2]); System.out.println("----------");
            found = true;
        }
    }

    if (!found) {
        System.out.println(" No matching notes found.");
    }

    br.close();
}

// Delete Note
public static void deleteNote(Scanner sc) throws IOException
    { File file = new File(FILE_NAME);

    if (!file.exists()) {
        System.out.println("No notes
        found."); return;
    }

    System.out.print("Enter Note ID to delete: ");
    int deleteId = sc.nextInt();
```

```
BufferedReader br = new BufferedReader(new FileReader(file));
File tempFile = new File("temp.txt");
BufferedWriter bw = new BufferedWriter(new FileWriter(tempFile));

String line;
boolean found = false;

while ((line = br.readLine()) != null)
    { String[] parts = line.split("\\|");

    if (Integer.parseInt(parts[0]) != deleteId)
        { bw.write(line + "\n");
    } else {
        found = true;
    }
}

br.close();
bw.close();

file.delete();
tempFile.renameTo(file);

if (found) {
    System.out.println(" Note deleted successfully!");
} else {
    System.out.println("Note not found.");
}
```

```
}

// Main Menu
public static void main(String[] args) throws IOException
   { Scanner sc = new Scanner(System.in);

   while (true) {
      System.out.println("\n===== Digital Note Organizer =====");
      System.out.println("1. Add Note");
      System.out.println("2. View Notes");
      System.out.println("3. Search Notes");
      System.out.println("4. Delete Note");
      System.out.println("5. Exit");
      System.out.print("Choose option: ");

      int choice = sc.nextInt();
      sc.nextLine();

      switch (choice)
         { case 1:
            addNote(sc);
            break;
         case 2:
            viewNotes();
            break;
         case 3:
            searchNotes(sc);
            break;
         case 4:
```

```
                deleteNote(sc);
                break;
            case 5:
                System.out.println(" Exiting...");
                return;
            default:
                System.out.println("Invalid choice!");
        }
    }
}
}
```

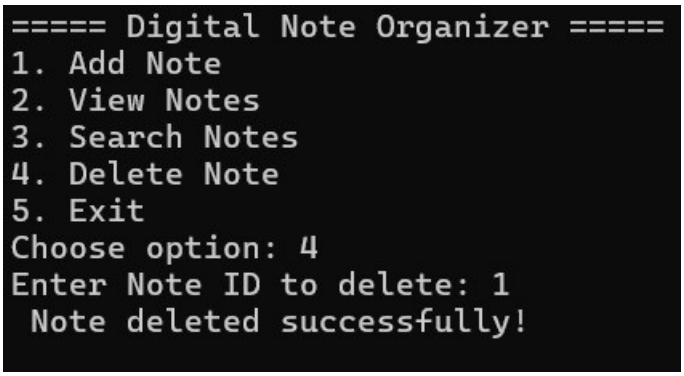
6. INPUT AND OUTPUT SCREENS

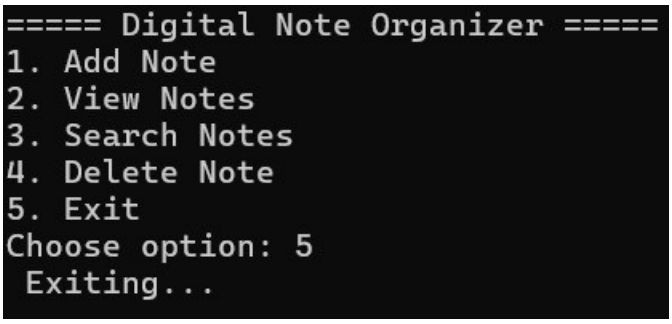
```
===== Digital Note Organizer =====
1. Add Note
2. View Notes
3. Search Notes
4. Delete Note
5. Exit
Choose option: 1
Enter Note ID: 1
Enter Title: Java
Enter Content: Study OOP Concept
Note added successfully!
```

```
===== Digital Note Organizer =====
1. Add Note
2. View Notes
3. Search Notes
4. Delete Note
5. Exit
Choose option: 2

All Notes:
ID: 1
Title: Java
Content: Study OOP Concept
----------------------
```

```
===== Digital Note Organizer =====
1. Add Note
2. View Notes
3. Search Notes
4. Delete Note
5. Exit
Choose option: 3
Enter keyword to search: 1
ID: 1
Title: Java
Content: Study OOP Concept
----------------------
```


===== Digital Note Organizer =====
1. Add Note
2. View Notes
3. Search Notes
4. Delete Note
5. Exit
Choose option: 4
Enter Note ID to delete: 1
Note deleted successfully!


===== Digital Note Organizer =====
1. Add Note
2. View Notes
3. Search Notes
4. Delete Note
5. Exit
Choose option: 5
Exiting...

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Digital Note Organizer with File Storage was developed to provide a simple and effective solution for managing digital notes. The system allows users to create, store, view, and delete notes easily using a menu-driven interface.

During the development of this project, various Core Java programming concepts such as classes, methods, loops, conditional statements, and file handling were implemented. These concepts helped in building a structured program that performs note management operations efficiently.

The system was tested with different inputs to verify that notes can be created, stored, viewed, and deleted correctly. The results show that the application works properly and helps users organize their notes in a simple digital format. By using file storage, the system ensures that notes remain saved even after the program is closed.

7.2 Conclusion

The Digital Note Organizer with File Storage successfully demonstrates the practical use of Java programming in developing a simple real-world application. The system helps users manage their notes efficiently by providing options to create, store, view, and delete notes.

The project reduces the difficulty of organizing information and provides an easy method for storing important notes digitally. It also helps students understand how Java programming concepts can be applied to develop useful applications.

In the future, the system can be improved by adding features such as a graphical user interface, database connectivity, search functionality, and cloud storage support. These enhancements would make the application more powerful and user-friendly.

8. REFERENCES

The following resources were referred to during the development of this project.

Books

• Herbert Schildt – *Java: The Complete Reference*
• Kathy Sierra & Bert Bates – *Head First Java*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com